

UNIVERSIDAD DE GRANADA

Arquitecturas y Computación de Altas Prestaciones

Practica 2 Parte 1

Ignacio Palomares Martín

1. Introducción y Objetivos

En esta primera tarea de la Práctica 2, el objetivo ha sido coger el código base que calculaba un histograma en CUDA y mejorarlo. El código original funcionaba, pero era muy poco eficiente porque todas las hebras intentaban escribir a la vez en la memoria global. En esta memoria voy a explicar los cambios que he ido haciendo en el código para solucionar esto y cómo ha mejorado el rendimiento al medir y comparar los tiempos.

2. Decisiones de Implementación

Para conseguir que el programa rinda mejor y cumpla con lo que pide el guión, he tomado un par de decisiones importantes a la hora de programar el kernel:

- **Soporte para cualquier tamaño n:** El código de partida tenía la pega de que solo funcionaba bien si la imagen era cuadrada y su tamaño era un múltiplo exacto del tamaño del bloque. Para arreglar esto y que acepte cualquier n, he cambiado la forma de calcular el *Grid* al lanzar el kernel, usando una división con redondeo hacia arriba: (n + blockSize.x - 1) / blockSize.x. Además, dentro del kernel, he metido un if (ix < n && iy < n) para asegurarme de que las hebras que caen fuera de los límites de la imagen no hagan nada y no tiren un error de acceso a memoria. Para conseguir esto en el programa principal (`main`), el cálculo de las dimensiones del *Grid* quedó así:

```
// Puedes probar BLOCK_SIZE en 8, 16 o 32 para comparar
int BLOCK_SIZE = 16;
dim3 blockSize(BLOCK_SIZE, BLOCK_SIZE);

// Asi calculamos el grid para que funcione con cualquier n
dim3 gridSize((n + blockSize.x - 1) / blockSize.x, (n + blockSize.y - 1) / blockSize.y);
```

Y dentro del *kernel*, calculamos las coordenadas globales del hilo y aplicamos la guarda de seguridad antes de leer la memoria:

```
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix < n && iy < n) {
        uint8_t pixelValue = d_image[iy * n + ix];
        atomicAdd(&hist_local[pixelValue], 1);
    }
    __syncthreads();

    if (tid < 256) {
        atomicAdd(&d_histogram[tid], hist_local[tid]);
    }
}
```

- **Uso de memoria compartida:** Para evitar el tremendo cuello de botella que supone usar la memoria global para todo , he tirado de memoria compartida (__shared__). Básicamente, he creado un array de 256 posiciones exclusivo para cada bloque. Al empezar, las primeras 256 hebras del bloque ponen su array a cero. Luego usamos un __syncthreads() para esperar. A partir de ahí, cada hebra lee su píxel y hace el atomicAdd sobre ese array rápido del bloque, en lugar de ir a la memoria global. Cuando todas terminan (con otro __syncthreads()), volcamos los resultados de ese bloque directamente en la memoria global. La implementación de esta caché local y las barreras de sincronización en el *kernel* se estructuró de la siguiente manera:

```
// Kernel en GPU usando memoria compartida para que vaya mas rapido
__global__ void computeHistogramShared(uint32_t *d_histogram, const uint8_t *d_image, const int n) {
    __shared__ uint32_t hist_local[256];

    int tid = threadIdx.y * blockDim.x + threadIdx.x;

    if (tid < 256) {
        hist_local[tid] = 0;
    }
    __syncthreads();

    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix < n && iy < n) {
        uint8_t pixelValue = d_image[iy * n + ix];
        atomicAdd(&hist_local[pixelValue], 1);
    }
    __syncthreads();

    if (tid < 256) {
        atomicAdd(&d_histogram[tid], hist_local[tid]);
    }
}
```

3. Estudio de Tiempos y Rendimiento

Para comprobar si esto de verdad sirve de algo y hacer el estudio de tiempos , he programado una versión secuencial en CPU normal y corriente para tenerla como referencia. Para medir los milisegundos exactos que tarda la GPU, he usado la API de eventos de CUDA (cudaEvent_t).

3.1. Impacto del tamaño de bloque

Primero, he querido comprobar cómo afecta cambiar las dimensiones del bloque manteniendo una imagen grande y fija (n = 4096).

Tamaño de Imagen (n)	Tamaño de Bloque	Hilos por Bloque	Tiempo CPU (ms)	Tiempo GPU Total (ms)	Tiempo GPU Solo Kernel (ms)
4096	**8×8**	64	25,553	3,905	2,056

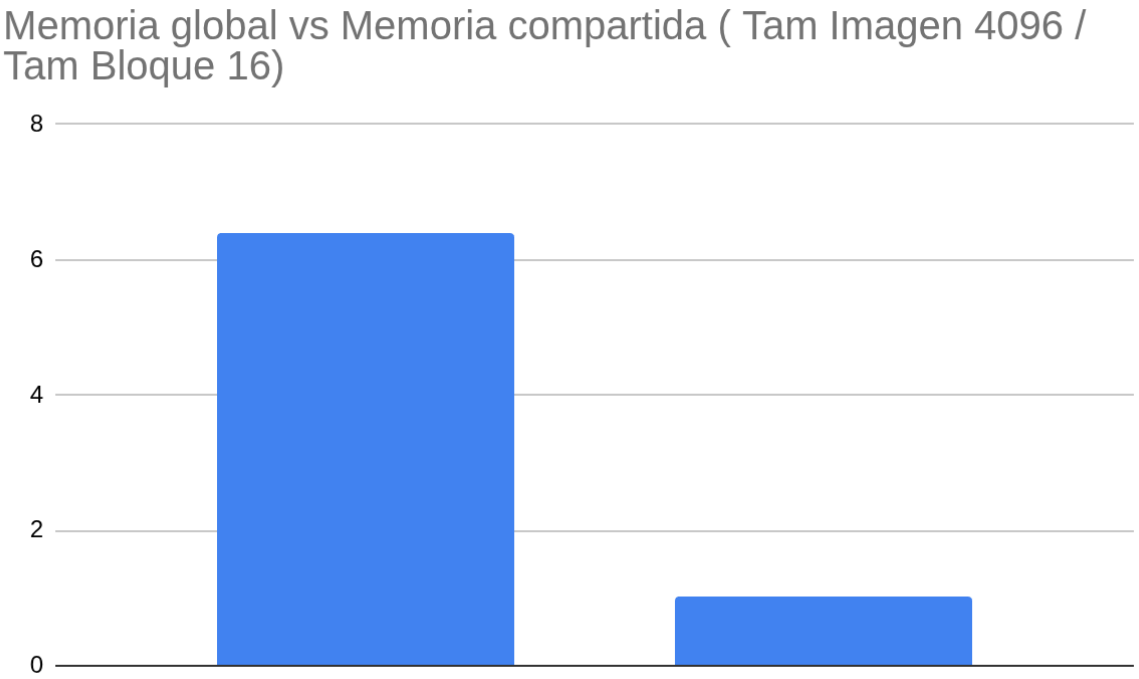
4096	**16×16**	256	25,541	2,842	1,034
4096	**32×32**	1024	25,213	2,165	0,272

Como podemos ver en los datos, el tamaño del bloque importa bastante. Con bloques pequeños de **8 x 8** (solo 64 hebras), estamos desaprovechando los recursos de los multiprocesadores (SM). Sin embargo, al probar con bloques de **32 x 32** (1024 hebras), el tiempo del kernel se desploma a apenas 0.272 ms. Esto pasa porque al tener tantas hebras por bloque, la tarjeta gráfica puede ir alternando entre distintos *warps* de forma mucho más eficiente, ocultando los tiempos de espera cuando se lee de la memoria.

3.2. Impacto de la memoria compartida

Aquí la idea era ver la diferencia real entre el código ingenuo que nos daban (solo memoria global) y el mío ya optimizado.

Versión del Algoritmo	Tamaño de Imagen (n)	Tamaño de Bloque	Tiempo GPU Solo Kernel (ms)
Memoria Global (Sin optimizar)	4096	16	6,388
Memoria Compartida (Optimizada)	4096	16	1,034



La diferencia es abismal. En el código original, miles de hebras se pelean continuamente por escribir en las mismas 256 posiciones usando atomicAdd. Al introducir la memoria compartida, ese problema desaparece casi por completo. Cada bloque hace sus cuentas tranquilamente en su propia caché local ultrarrápida y solo molesta a la memoria global una vez al final para sumar sus resultados.

3.3. Análisis de Aceleración (Speedup)

Por último, he calculado la aceleración obtenida frente a la CPU , probando con tamaños de imagen cada vez mayores y midiendo con y sin copias de memoria.

Tamaño de Imagen (n=32)	Tiempo CPU (ms)	Tiempo GPU Total (ms)	Tiempo GPU Solo Kernel (ms)	Speedup Total (CPU / GPU Total)	Speedup Kernel (CPU / GPU Kernel)
512	0,391	0,188	0,009	2.08x	45.25x
1024	1,574	0,270	0,021	5.84x	75.67x
2048	6,277	0,719	0,070	8.73x	90.15x
4096	25,461	2,138	0,272	11.91x	93.49x
8192	102,592	7,893	1,075	13.00x	95.43x

Al sacar estos datos, me di cuenta de algo clave en la arquitectura. Si miramos solo lo que tarda en ejecutarse el kernel puro y duro, el *Speedup* es enorme (pasando de 90x para las imágenes más grandes). La GPU calcula rapidísimo. Sin embargo, si miramos el "*Speedup* Total", la aceleración baja muchísimo (a unos 13x). La razón es muy sencilla: el tiempo que gasta el sistema en copiar la imagen de la RAM a la VRAM y luego devolver el histograma con cudaMemcpy es altísimo y se convierte en el verdadero cuello de botella de nuestro programa.

3.4. Análisis Estadístico y Estabilidad de las Mediciones

Para asegurar el rigor de este estudio de rendimiento, no he tomado una única medición para los tiempos, sino que he realizado 5 ejecuciones consecutivas para cada tamaño de imagen (utilizando la configuración de bloque de 16x16) y he calculado su media y desviación típica muestral. Esto permite ver cómo de estables son los tiempos en la tarjeta gráfica.

Tamaño imagen (bloque n=16)	512	
N ejecución	Tiempo GPU Total	Tiempo GPU shared (solo kernel)
1	0,192	0,021
2	0,194	0,020
3	0,183	0,020
4	0,188	0,020
5	0,190	0,020
Media	0,1894	0,0202
Desviación Típica	0,004219004622	0,0004472135955

Tamaño imagen (bloque n=16)	1024	
N ejecución	Tiempo GPU Total	Tiempo GPU shared (solo kernel)
1	0,326	0,068
2	0,320	0,068
3	0,314	0,069
4	0,315	0,069
5	0,315	0,068
Media	0,318	0,0684
Desviación Típica	0,005049752469	0,0005477225575

Tamaño imagen (bloque n=16)	2048	
N ejecución	Tiempo GPU Total	Tiempo GPU shared (solo kernel)
1	0,905	0,262
2	0,911	0,262
3	0,914	0,262
4	0,908	0,262

5	0,915	0,262
Media	0,9106	0,262
Desviación Típica	0,004159326869	0

Tamaño imagen (bloque n=16)	4096	
N ejecución	Tiempo GPU Total	Tiempo GPU shared (solo kernel)
1	2,828	1,034
2	2,829	1,034
3	2,853	1,034
4	2,859	1,033
5	2,844	1,034
Media	2,8426	1,0338
Desviación Típica	0,01393915349	0,0004472135955

Tamaño imagen (bloque n=16)	8192	
N ejecución	Tiempo GPU Total	Tiempo GPU shared (solo kernel)
1	10,669	4,118
2	11,129	4,117
3	11,691	4,118
4	10,667	4,118
5	10,932	4,119
Media	11,0176	4,118
Desviación Típica	0,4235703011	0,0007071067812

Al analizar estos datos estadísticos, se observan dos comportamientos arquitectónicos muy distintos que confirman la teoría:

1. **La precisión determinista del Kernel:** Si nos fijamos en la desviación típica del tiempo del kernel puro (Tiempo GPU shared), vemos que es prácticamente nula. Incluso para la imagen más masiva de 8192x8192, la desviación es de apenas **0,0007 ms**. Esto nos demuestra que una vez que el kernel está planificado en los *Streaming Multiprocessors* (SM), la ejecución matemática y el acceso a la memoria compartida son procesos extremadamente estables y predecibles. El hardware ejecuta las instrucciones como un reloj.
2. **La variabilidad del bus del sistema:** Por otro lado, la desviación típica del "Tiempo GPU Total" es un orden de magnitud mayor. En el caso de la imagen de 8192x8192, la desviación salta a **0,423 ms**. Esta fluctuación ocurre porque este tiempo incluye las llamadas cudaMemcpy. Las transferencias por el bus PCI-Express dependen de la carga del sistema operativo en ese instante, de la gestión de la memoria RAM del *Host* y de la disponibilidad del propio bus, lo que introduce un factor de imprevisibilidad y ruido en las mediciones.

En conclusión, este análisis demuestra que la computación en la GPU es enormemente estable, y que la mayor fuente de variabilidad e inestabilidad en los tiempos de ejecución proviene siempre de la comunicación entre la CPU y la GPU.

4. Conclusiones

Hacer esta primera tarea me ha servido para comprobar que programar en CUDA no consiste solo en lanzar muchísimas hebras y ya está. Para que un código paralelo rinda bien, es fundamental entender y gestionar bien la jerarquía de memorias, aprovechando al máximo memorias rápidas como la compartida. Además, queda muy claro que, aunque tengamos una GPU muy potente calculando, siempre vamos a estar limitados por lo lentas que son las transferencias de datos por el bus del sistema.